

# Meet Leverage

Your expensive model just got a team.



Built for the RocketRide x SCU Buildathon

Cloud workers run as RocketRide pipelines. Credits read from billing.

# It is 2 a.m. The suite is red.

You paste the failing test into your best model. Third time tonight. It fixes one thing and breaks another. Then the rate limit lands, and you sit there waiting for a quota to reset.

Everyone who has shipped with an AI in the loop knows this night.

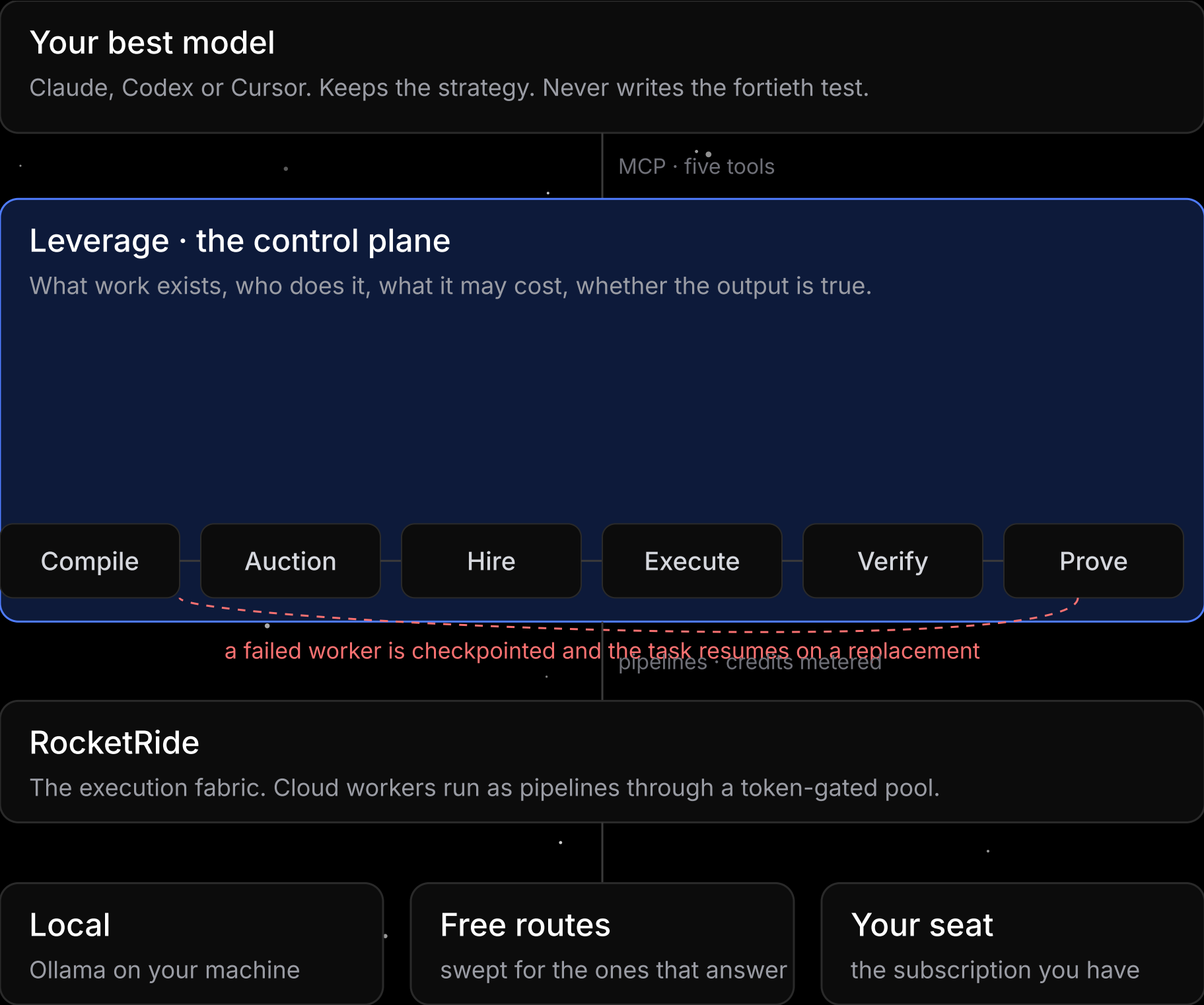
# What if the smart one delegated?

Leverage sits under the model you already pay for. That model keeps the plan. A team of cheaper models takes the grind: the boilerplate, the retries, the test that has to go green.

|         |                                     |
|---------|-------------------------------------|
| Decides | your frontier model                 |
| Does    | local, free and subscription models |
| Judges  | the repository's own tests          |

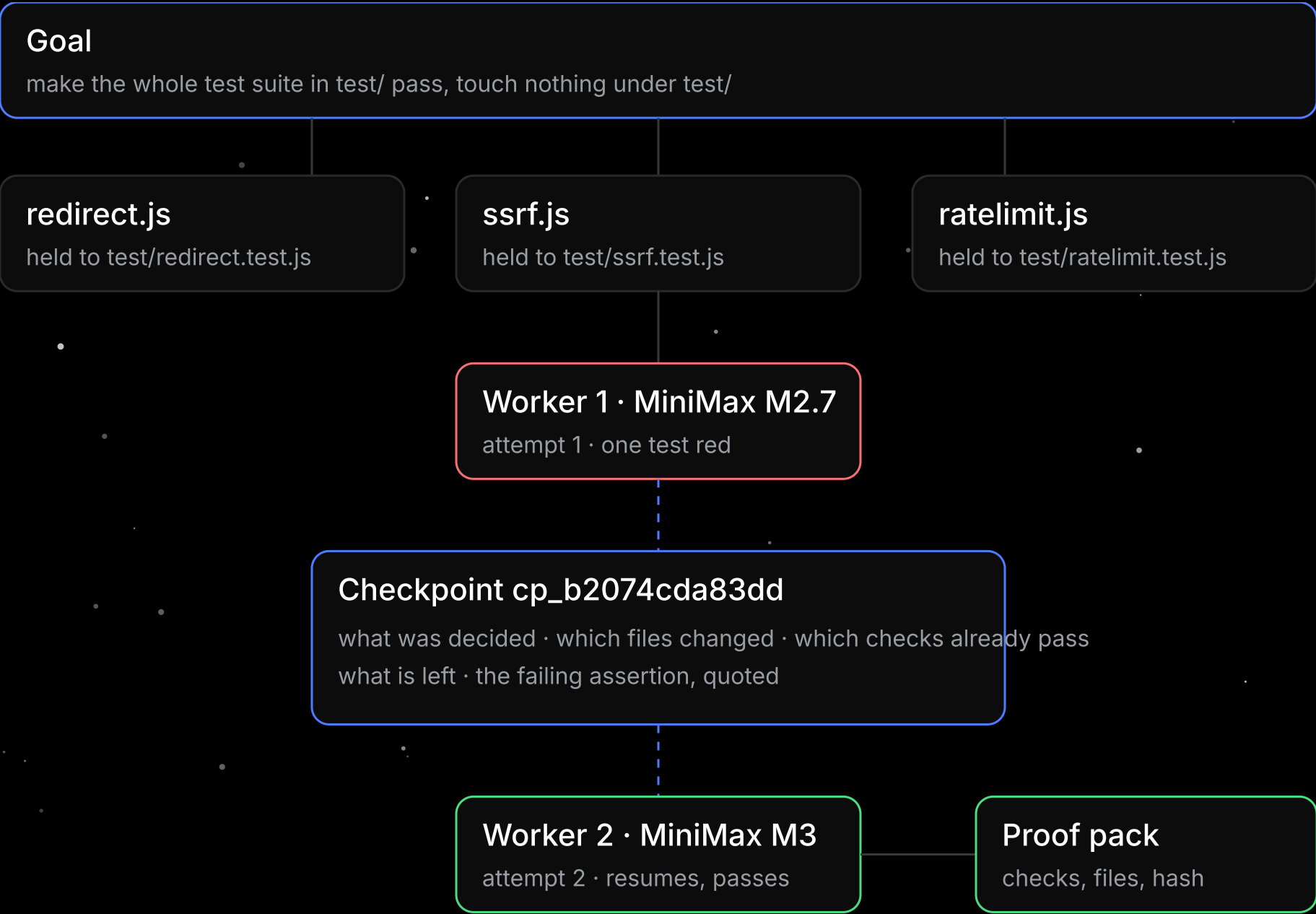
# The harness.

Six stages between a goal and a proof, and one loop that runs when something breaks.



# A worker dies. The work does not.

When a model hits a limit or fails a test, Leverage writes down what it understood and hands that note to the next one. Nobody starts from zero.



One of the two handoffs in mission LVR-e6443739. The second worker never re-read the whole repository.

# The tests are the boss.

A task is finished when the test runner says so, not when a model says so. After the last task, the whole suite runs once more. Everything that passed is written into a proof pack you can read.

---

|        |                   |
|--------|-------------------|
| Exit 0 | or it is not done |
|--------|-------------------|

---

|             |                       |
|-------------|-----------------------|
| Whole suite | runs again at the end |
|-------------|-----------------------|

---

|            |                            |
|------------|----------------------------|
| Proof pack | checks, files, hash, spend |
|------------|----------------------------|

---

# Tonight it built a chat product.

We wrote the tests for a Claude-style assistant: conversations, a context window, rate limits, a model adapter, an HTTP server and the page. Then one message. This is Quill, running against a model on the laptop.

## Quill Chat

In two short sentences, what is a sliding window rate limiter?

A sliding window rate limiter is a technique for limiting the delivery of a sequence of HTTP requests by counting requests in a rolling time window and enforcing a maximum number of requests that can be made during that period.

Type a message...

Send



# Then it wrote a paper.

A position paper on delegating work to AI agents, held to tests: the sections, the length, six numbered references, and an evidence section that may only quote what a running system recorded. No invented experiments allowed, and the test checks for that too.

## Verification Without Trust

Position paper · written by a Leverage worker model · held to test/paper.test.js · September 2026

### ABSTRACT

The delegation problem emerges when autonomous systems entrust critical tasks to subcomponents without direct oversight. This paper examines how verification functions as the contractual foundation for reliable delegation, using evidence from a running system to demonstrate practical constraints. We analyze handoff context degradation, planning latency, cost structures, and proof compliance across multiple mission phases. The evidence shows context carried into replacement workers shrank by 57%, 48% and 30% across three handoffs in one recorded mission, while a planner model wrote a three-task graph from a repository's test files in 45.7 seconds. Paid inference across five recorded missions was \$0.00, and a four-task mission passed 8 of 8 proof checks and a 17-test suite with exit code 0. One chat message produced three security guards that passed their own tests on the second attempt after two worker replacements. This paper frames verification not as a technical add-on but as the essential contract governing delegation, identifies boundary conditions through empirical observation, and outlines limitations for future work. The analysis provides concrete metrics for system designers confronting autonomous delegation decisions. Keywords: delegation, verification, autonomous systems, empirical evidence

verification conditions, or delegation must be suspended until proof is re-established. This creates a natural boundary: delegation is not a one-time handoff but an ongoing contractual relationship that requires continuous verification. The economic evidence shows that this relationship can be maintained at zero marginal cost, but the contractual obligation remains.

A critical aspect of the verification contract is its enforceability. If a worker fails to produce the required proof, the delegating agent must have a predefined response such as rolling back the task, assigning a replacement worker, or flagging the mission for human review. The evidence from a running system shows that after two worker replacements, security guards produced by an initial chat message passed their own tests on the second attempt, demonstrating that the contractual mechanism of proof generation and inspection can recover from intermediate failures. This recovery is not automatic; it depends on the verification contract being explicitly defined and checked at each delegation point.

### EVIDENCE FROM A RUNNING SYSTEM

The following metrics were recorded during actual mission execution using the Leverage system. Each fact represents an observed phenomenon rather than



# It shows its failures.

The first pass on Quill: three of six modules green, fifteen workers hired, every failed one named with its checkpoint. Two short follow-up missions finished the rest. The console never hides a red task, and that is the point.

Leverage

Overview

New mission

Live run

Missions

Models

Providers

MISSION LVR-5441F44D

Build Quill, a Claude-style assistant chat product, so that the whole test suite in test/ passes: src/conversations.js, src/context.js, src/limits.js, src/provider.js, src/server.js and public/index.html. Node built-ins only, CommonJS modules. Do not modify any file under test/. The chat page should look like a polished, minimal assistant chat: a message list, a composer with a textarea and a send button, inline styles, and a script that creates a conversation and posts messages to the API.

plan written by Nemotron 3 Nano 30B via NVIDIA (free) · 6 tasks in 100.1s

event stream complete · budget \$0.00 hard · privacy prefer-local

|         |            |         |       |                |                            |
|---------|------------|---------|-------|----------------|----------------------------|
| QUALITY | PAID SPEND | ELAPSED | TASKS | ACTIVE WORKERS | FRONTIER-EQUIV (estimated) |
| 100     | \$0.00     | 930.7s  | 3/6   | 0              | \$0.2382                   |

TASK GRAPH

INTEGRATION

Implement conversation store

FAILED attempt 4

BACKEND

Implement prompt builder

PASSED

BACKEND

Implement rate limiter and token budget

FAILED attempt 4

BACKEND

Implement fetch wrapper

PASSED

BACKEND

Implement HTTP server

FAILED attempt 4

FRONTEND

Implement chat page UI

PASSED attempt 2

WORKFORCE

Backend Engineer

REPLACED

qwen2.5-coder:3b local runtime

100% task fit · 27% verified success over 24 prior jobs · local runtime, no spend · 2 candidates excluded by policy

context 3728 tok \$0.0000

resumed from cp\_cddb3fe1be24 TEST\_FAILURE

Backend Engineer

REPLACED

MiniMax M3 via NVIDIA RocketRide pipeline

100% task fit · 37% verified success over 8 prior jobs · free route, no spend · 2 candidates excluded by policy

context 3714 tok \$0.0000

resumed from cp\_4501d5415988 INVALID\_OUTPUT

Frontend Engineer

PASSED

Nemotron 3 Super 120B via NVIDIA RocketRide p...

61% task fit · no prior jobs, scored on the neutral prior · free route, no spend

context 488 tok \$0.0000 resumed from cp\_f2501e8bf265

PLAN

written by a model

# Things people hand to it.

From the missions it has actually run.

---

## The failing suite before standup

a four-task repo, seventeen tests, three worker failures survived

---

## The physics for a small game

vector maths, a seeded spawner, a state machine, 22 tests green

---

## A chat product from a test spec

six modules, one mission, running against a local model the same night

---

## A paper that may not lie

structure, references and honesty checked by tests before anyone reads it

---

# It costs what you already pay.

Local models on your laptop. Free hosted routes. The subscription seat you have. Set the budget to zero and paid routes are not discouraged, they are removed from the room before the auction starts.

---

**Local**

Ollama, on your machine

---

**Free**

hosted routes, swept for the ones that answer

---

**Yours**

the seat you already pay for

---

# Press the button.

A real mission runs on the live site while you watch. Planned by a model, verified by its tests.

[useleverage.vercel.app](https://useleverage.vercel.app)